

# GPU Programming: Assignment Solutions

## Comprehensive Pedagogical Guide & In-Depth Derivations

Peteti Charan Tej

Roll No : g25ait2026 | Course: GPU Programming & Accelerated Computing

September 20, 2026

### Abstract

This document provides the complete numerical solutions, thorough architectural explanations, and step-by-step mathematical derivations for the 10 GPU programming questions. Each question is structured with: (1) **Core Concept and Hardware Context**, (2) **Step-by-Step Walkthrough and Calculation**, (3) **Common Student Pitfalls**, and (4) **Final Answer Box Value**.

### Quick-Reference Master Answer Key

| Q#  | Box Answer | Concept Tested            | Summary Description & Formula                                                                              |
|-----|------------|---------------------------|------------------------------------------------------------------------------------------------------------|
| Q1  | 2          | Compute Throughput        | Peak FP32 = $4 \text{ FMA} \times 2 \text{ FLOPs} \times 1.5 \text{ GHz} = 12 \text{ GFLOPS}$ (10.5–12.5). |
| Q2  | 524288     | Global Memory Traffic     | Naive matmul: $256 \text{ threads} \times (1024_A + 1024_B) = 524,288$ global loads.                       |
| Q3  | 32768      | Tiling & Shared Memory    | Tiled matmul: $64 \text{ phases} \times 256 \text{ threads} \times 2 \text{ loads} = 32,768$ global loads. |
| Q4  | 3          | Race Conditions / Hazards | Both (a) is true (RAW hazard) and (b) is true (WAR hazard).                                                |
| Q5  | 32         | Memory Coalescing         | Column-major store: stride is 8192 bytes $\gg$ 128 bytes $\implies$ 32 segments.                           |
| Q6  | 5          | CUDA Qualifiers           | Kernel function declaration must be <code>__global__ void</code> .                                         |
| Q7  | 13         | Matrix-Vector Product     | Thread $i = 2$ : $y[2] = (2 \times 1) + (0 \times 2) + (1 \times 3) + (2 \times 4) = 13$ .                 |
| Q8  | 1          | Memory Hierarchy          | <b>Global memory</b> is accessible by all SMs and persists across application.                             |
| Q9  | 120        | SIMT Warp Divergence      | Threads where $\text{tid} \pmod{4} = 1$ : $\{1, 5, 9, 13, 17, 21, 25, 29\}$ ; sum is 120.                  |
| Q10 | 32768      | Grid & Warp Sizing        | $N = 2^{20}$ , $4096 \text{ blocks} \times 8 \text{ warps/block} = 32,768$ total warps launched.           |

## Detailed Solutions and Pedagogical Explanations

### Question 1: Peak FP32 Throughput of a Streaming Multiprocessor (SM)

#### Core Concepts: What is an SM, FMA, and FLOP?

**Streaming Multiprocessor (SM):** The primary compute engine of an NVIDIA GPU containing CUDA cores, warp schedulers, register files, and cache.

**Fused Multiply-Add (FMA):** An instruction performing  $a \times b + c$  as a single atomic operation with a single rounding step.

**Why 1 FMA = 2 FLOPs?** It carries out two distinct arithmetic operations: one floating-point multiplication and one floating-point addition. Thus, 1 FMA instruction = 2 floating-point operations (FLOPs).

#### Problem Context:

- Total SMs on GPU = 80. (Note: Problem asks specifically for a **single SM**).
- CUDA cores per SM = 64.
- Sustained execution: Every cycle, each SM achieves a peak of 4 FP32 FMA operations per cycle.
- Core clock frequency  $f = 1.5 \text{ GHz} = 1.5 \times 10^9 \text{ cycles per second}$ .

#### Step-by-Step Mathematical Derivation:

##### 1. Calculate operations per clock cycle per SM:

$$\text{FLOPs per cycle} = (\text{FMA operations per cycle}) \times (\text{FLOPs per FMA})$$

$$\text{FLOPs per cycle} = 4 \times 2 = 8 \text{ FLOPs/cycle.}$$

##### 2. Calculate operations per second (Throughput in GFLOPS):

$$\begin{aligned} \text{Peak Throughput} &= \text{FLOPs per cycle} \times \text{Clock Frequency} \\ &= 8 \text{ FLOPs/cycle} \times 1.5 \times 10^9 \text{ cycles/second} \\ &= 12 \times 10^9 \text{ FLOPs/second} = \mathbf{12 \text{ GFLOPS}}. \end{aligned}$$

##### 3. Map to the specified multiple-choice ranges:

- Range 1: 8 to 10 GFLOPS
- Range 2: 10.5 to 12.5 GFLOPS  $\implies$  **12 GFLOPS falls exactly inside this range!**
- Range 3: 12.5 to 15 GFLOPS
- Range 4: 15 to 20 GFLOPS

#### Student Pitfall Alert:

- **Pitfall 1:** Multiplying by 80 SMs.  $80 \times 12 = 960 \text{ GFLOPS}$ . The question explicitly asks for the peak throughput of a *single SM*.
- **Pitfall 2:** Forgetting that each FMA equals 2 FLOPs, which would give an incorrect 6 GFLOPS.

#### Final Answer to Enter in Box: 2

*The peak FP32 throughput of a single SM is 12 GFLOPS, which falls between 10.5 and 12.5 GFLOPS.*

## Question 2: Naive Matrix Multiplication Global Memory Traffic

### Core Concepts: The Cost of Redundant Global Memory Accesses

In a naive matrix multiplication  $C = A \times B$ , every output element  $C[\text{row}, \text{col}]$  is computed by one thread executing a dot product between row  $\text{row}$  of  $A$  and column  $\text{col}$  of  $B$ . Because the GPU's off-chip global memory has high latency ( $\sim 400$  clock cycles) and limited bandwidth, loading every single scalar operand directly from DRAM creates massive memory bottlenecks.

### Kernel Code Under Analysis:

```

1 __global__ void matMulNaive(const float *A, const float *B, float *C, int N) {
2     int row = blockIdx.y * blockDim.y + threadIdx.y;
3     int col = blockIdx.x * blockDim.x + threadIdx.x;
4     if (row < N && col < N) {
5         float sum = 0.0f;
6         for (int k = 0; k < N; k++) {
7             sum += A[row * N + k] * B[k * N + col]; // Loads from global memory
8         }
9         C[row * N + col] = sum;
10    }
11 }

```

### Step-by-Step Derivation:

#### 1. Determine the number of active threads per block:

- Block size is  $16 \times 16 \implies$  Total threads per block =  $16 \times 16 = 256$  threads.
- $N = 1024$ . Since 1024 is evenly divisible by 16 ( $1024/16 = 64$  blocks in each dimension), there are no boundary clipping issues. Every thread satisfies ( $\text{row} < N$  and  $\text{col} < N$ ) and is fully active.

#### 2. Determine memory loads per thread:

- The loop executes  $k = 0, 1, \dots, N - 1$  ( $N = 1024$  iterations).
- In *every* iteration, the thread loads:

1 float from array  $A$  ( $A[\text{row} * N + k]$ )

1 float from array  $B$  ( $B[k * N + \text{col}]$ )

- Loads per thread from  $A = 1024$  loads.
- Loads per thread from  $B = 1024$  loads.
- Total global memory loads per thread =  $1024 + 1024 = 2048$  loads.

#### 3. Calculate total loads per block:

$$\text{Total Loads} = (\text{Threads per Block}) \times (\text{Loads per Thread})$$

$$\text{Total Loads} = 256 \times 2048 = 2^8 \times 2^{11} = 2^{19} = \mathbf{524,288} \text{ loads.}$$

**Why is this so inefficient?** Notice that all 16 threads in the same row of the block need the exact same row of  $A$ , yet each thread loads it independently from global memory 16 separate times! Similarly, all 16 threads in the same column load the same column of  $B$  16 separate times. This redundant traffic motivates the tiled version in Q3.

**Final Answer to Enter in Box: 524288**

*Exact total number of global memory loads from matrix  $A$  and matrix  $B$  per block.*

### Question 3: Tiled Matrix Multiplication Memory Traffic Reduction

#### Core Concepts: Why Shared Memory Tiling Improves Performance

Shared memory is on-chip SRAM located directly inside each SM. It is nearly as fast as registers ( $\sim 1-5$  cycles latency vs  $\sim 400$  cycles for DRAM). In tiled matrix multiplication, threads in a block collaborate to load a small  $16 \times 16$  tile of  $A$  and  $B$  into shared memory once. The threads then reuse the cached values 16 times from fast on-chip memory, slashing global memory traffic by a factor of 16.

#### Step-by-Step Derivation:

##### 1. Identify the dimensions:

- Matrix dimension  $N = 1024$ .
- Tile size =  $16 \times 16$ .
- Block size =  $16 \times 16 = 256$  threads.

##### 2. Calculate the number of phases (tile iterations):

The outer loop steps across matrix  $A$  horizontally and matrix  $B$  vertically in chunks of 16:

$$\text{Number of phases} = \frac{N}{\text{Tile Dimension}} = \frac{1024}{16} = 64 \text{ phases.}$$

##### 3. Calculate global loads per phase for the block:

In each phase  $k$ :

- Each thread loads 1 element of  $A$  into shared memory: `tileA[ty][tx] = A[...];`
- Each thread loads 1 element of  $B$  into shared memory: `tileB[ty][tx] = B[...];`
- Each thread issues  $1 + 1 = 2$  global loads per phase.
- Across all 256 threads in the block:

$$\text{Loads per Phase per Block} = 256 \text{ threads} \times 2 \text{ loads/thread} = 512 \text{ loads.}$$

##### 4. Calculate total global loads per block across all 64 phases:

$$\text{Total Loads} = 64 \text{ phases} \times 512 \text{ loads/phase} = 2^6 \times 2^9 = 2^{15} = \mathbf{32,768} \text{ loads.}$$

| Implementation                   | Global Loads per Block | Global Memory Traffic Reduction          |
|----------------------------------|------------------------|------------------------------------------|
| Naive Matrix Multiplication (Q2) | 524,288                | $1 \times$ (Baseline, unoptimized)       |
| Tiled Matrix Multiplication (Q3) | 32,768                 | $16 \times$ reduction (Bandwidth saved!) |

**Direct Comparison: Naive vs. Tiled:** The factor of reduction is exactly equal to the tile width:  
 $\frac{524288}{32768} = 16$ .

#### Final Answer to Enter in Box: 32768

*Total number of global memory loads per block in the tiled version.*

### Question 4: Race Conditions and Synchronization Hazards in CUDA

#### Core Concepts: What is `__syncthreads()` and Why is it Essential?

`__syncthreads()` acts as a barrier synchronization point for all threads in a thread block. No thread in the block can proceed past this barrier until all other threads have reached it. Furthermore, all shared and global memory accesses made prior to the barrier are guaranteed to be visible to all threads in the block after the barrier.

**The Buggy Kernel Code:**

```

1 for (int k = 0; k < N / 16; k++) {
2     // 1. Collaborative loading into shared memory
3     tileA[threadIdx.y][threadIdx.x] = A[row * N + (k * 16 + threadIdx.x)];
4     tileB[threadIdx.y][threadIdx.x] = B[(k * 16 + threadIdx.y) * N + col];
5     // MISSING: __syncthreads(); <-- Hazard (a) occurs here!
6
7     // 2. Computation from shared memory
8     for (int t = 0; t < 16; t++) {
9         sum += tileA[threadIdx.y][t] * tileB[t][threadIdx.x];
10    }
11    // MISSING: __syncthreads(); <-- Hazard (b) occurs here!
12 }

```

**Detailed Analysis of the Statements:**

1. **Statement (a): Thread (5,10) starts computation before thread (5,3) writes to tileA[5][3].**

- In the inner computation loop, thread (5,10) computes:

$$\text{sum} += \text{tileA}[5][t] \times \text{tileB}[t][10]$$

When  $t = 3$ , it reads the shared memory location `tileA[5][3]`.

- The thread responsible for writing into `tileA[5][3]` is thread (5,3).
  - Since threads execute in independent warps that are scheduled asynchronously without any guarantee of relative speed, warp scheduling can cause thread (5,10) to execute its read before thread (5,3) has executed its write!
  - This is a classic **Read-After-Write (RAW)** data race.
  - **Statement (a) is TRUE.**
2. **Statement (b): Thread (3,12) overwrites tileB[3][12] for the next iteration while another thread is still reading it.**
    - When the inner computation loop finishes, loop index  $k$  increments to  $k + 1$ .
    - In iteration  $k + 1$ , thread (3,12) immediately attempts to load a new float from global memory into `tileB[3][12]`.
    - However, because there is no barrier at the bottom of the loop, another thread (e.g., in a slower warp) might still be executing its inner loop for iteration  $k$ , relying on the *old* value of `tileB[3][12]`.
    - Overwriting this value before the other thread finishes reading causes a **Write-After-Read (WAR)** data race.
    - **Statement (b) is TRUE.**

**Conclusion:** Both statement (a) and statement (b) describe real, fatal race conditions caused by missing `__syncthreads()` barriers. Thus, option **3** is correct.

**Final Answer to Enter in Box: 3**

*Option 3: Statement (a) is true, and statement (b) is true.*

## Question 5: Global Memory Coalescing and 128-Byte Segments

### Core Concepts: How NVIDIA GPUs Access Global Memory

Global memory transactions are executed at the granularity of warps (32 threads). The GPU memory subsystem does not issue 32 independent 4-byte read/write requests. Instead, it combines (coalesces) concurrent memory requests from all active threads in a warp into as few cache lines / segments (128 bytes or 32 bytes) as possible.

#### Problem Context:

```

1 int x = bx * blockDim.x + tx;
2 int y = by * blockDim.y + ty;
3 if (x < width && y < height) {
4     float val = A[y * width + x]; // Row-major read
5     B[x * height + y] = val;      // Column-major store
6 }
```

- Array dimensions: width = height = 2048.
- Data type: float (4 bytes).
- Block structure: blockDim.x = 32.
- Warp composition: Each warp consists of 32 threads with consecutive  $tx \in \{0, 1, \dots, 31\}$ , meaning consecutive values of  $x = x_0, x_0 + 1, \dots, x_0 + 31$ , and a fixed value of  $y = y_0$ .

#### Mathematical Stride Derivation for the Store into $B$ :

1. The linear memory address in array  $B$  for thread  $tx$  is:

$$\text{Index}(tx) = x \times \text{height} + y = (x_0 + tx) \times 2048 + y_0.$$

2. Linear indices for consecutive threads:

$$\text{Thread } 0 : \text{ Index} = x_0 \times 2048 + y_0$$

$$\text{Thread } 1 : \text{ Index} = (x_0 + 1) \times 2048 + y_0 = x_0 \times 2048 + y_0 + 2048$$

$$\text{Thread } 2 : \text{ Index} = (x_0 + 2) \times 2048 + y_0 = x_0 \times 2048 + y_0 + 4096$$

$$\vdots$$

3. Stride in Elements:

$$\Delta \text{Index} = \text{Index}(tx + 1) - \text{Index}(tx) = 2048 \text{ elements.}$$

4. Stride in Bytes:

$$\text{Byte Stride} = 2048 \text{ elements} \times 4 \text{ bytes/element} = 8192 \text{ bytes.}$$

5. Count the 128-Byte Segments:

- Each 128-byte segment covers an aligned span of 128 contiguous bytes in memory.
- Since the distance between consecutive threads' access addresses is 8192 bytes  $\gg$  128 bytes, **no two threads in the warp can ever access memory inside the same 128-byte segment!**
- Therefore, each of the 32 threads in the warp demands its own distinct 128-byte memory transaction.
- Total 128-byte segments accessed = 32.

**Efficiency Impact for Students:**

- Total useful data written by the warp:  $32 \text{ threads} \times 4 \text{ bytes} = 128 \text{ bytes}$ .
- Total memory fetched/transferred across the bus:  $32 \text{ segments} \times 128 \text{ bytes/segment} = 4096 \text{ bytes}$ .
- Bus efficiency:  $\frac{128}{4096} = 3.125\%$ . This extreme striding illustrates why non-coalesced memory access severely degrades GPU performance.

**Final Answer to Enter in Box: 32**

*Exactly 32 distinct 128-byte memory segments are accessed by the warp.*

**Question 6: CUDA Function Execution Space Qualifiers****Core Concepts: The Three CUDA Function Qualifiers**

- `__global__`: Defines a **kernel function**. It is executed on the Device (GPU) and callable from the Host (CPU) or Device (via CUDA Dynamic Parallelism).
- `__device__`: Defines a helper function that executes on the Device and can **only** be called from the Device.
- `__host__`: Defines a normal CPU function that executes on the Host and is called from the Host (default if no qualifier is given).

**Why Must a Kernel Return void?**

- **Asynchronous Execution:** Kernel invocations using the execution configuration syntax (e.g., `<<<grid, block>>>`) return control to the host CPU thread immediately without waiting for device execution to finish.
- **Massive Concurrency:** A kernel is executed by thousands or millions of concurrent threads. There is no language mechanism or single scalar register to return a value from multiple threads to the host caller.
- Any data produced by a kernel must be stored explicitly into global memory via pointer arguments.
- Consequently, the CUDA compiler enforces that all `__global__` functions must have a return type of `void` (i.e., `__global__ void`).

**Evaluation of Question Options:**

- Option 1: `__host__ void`  $\Rightarrow$  Host function, not a device kernel.
- Option 2: `__shared__ char`  $\Rightarrow$  `__shared__` is a memory variable qualifier, not a function qualifier.
- Option 3: `__global__ float`  $\Rightarrow$  Syntax error in CUDA (kernels cannot return non-void types).
- Option 4: `__device__ int`  $\Rightarrow$  Device helper function, cannot be launched from host.
- Option 5: `__global__ void`  $\Rightarrow$  **Correct!**

**Final Answer to Enter in Box: 5**

*Option 5: `__global__ void`*

**Question 7: Matrix-Vector Multiplication for Thread  $i = 2$** **Core Concepts: Parallel Dot Product Mapping**

In matrix-vector multiplication  $y = A \cdot x$ , each row  $i$  of the matrix is multiplied with the vector  $x$  to yield scalar element  $y[i]$ :

$$y[i] = \sum_{j=0}^{N-1} A[i, j] \cdot x[j]$$

In CUDA, the standard parallel strategy assigns thread  $i$  to compute row  $i$  independently.

**Problem Parameters:**

- $N = 4$ , blockDim.x = 4, gridDim.x = 1.
- Inputs:

$$A = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 3 & 1 \\ 2 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \end{bmatrix}, \quad x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

**Step-by-Step Evaluation for Thread  $i = 2$ :**

1. Thread index calculation:

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x} = 0 \times 4 + 2 = 2.$$

2. Thread  $i = 2$  processes row index 2 of matrix  $A$  (0-indexed third row):

$$\text{Row 2 of } A = [A[2, 0], A[2, 1], A[2, 2], A[2, 3]] = [2, 0, 1, 2].$$

3. Vector  $x$ :

$$x = [x[0], x[1], x[2], x[3]]^T = [1, 2, 3, 4]^T.$$

4. Compute the dot product term by term:

$$j = 0 : A[2, 0] \times x[0] = 2 \times 1 = 2$$

$$j = 1 : A[2, 1] \times x[1] = 0 \times 2 = 0$$

$$j = 2 : A[2, 2] \times x[2] = 1 \times 3 = 3$$

$$j = 3 : A[2, 3] \times x[3] = 2 \times 4 = 8$$

5. Sum the terms:

$$y[2] = 2 + 0 + 3 + 8 = \mathbf{13}.$$

**Final Answer to Enter in Box: 13**

*The computed value of  $y[2]$  for thread  $i = 2$  is 13.*

**Question 8: CUDA Memory Spaces, Accessibility, and Persistence****Core Concepts: The CUDA Memory Hierarchy**

Different memory spaces on a GPU serve different computational purposes, with specific scopes and lifetimes.

| Memory Space           | Physical Location | Scope                    | Lifetime           | Key Characteristics                                            |
|------------------------|-------------------|--------------------------|--------------------|----------------------------------------------------------------|
| <b>Register</b>        | On-chip           | Single thread            | Thread             | Fastest access; holds private thread variables.                |
| <b>Shared Memory</b>   | On-chip           | Thread block             | Block              | High bandwidth; shared among threads in a block.               |
| <b>Local Memory</b>    | Off-chip DRAM     | Single thread            | Thread             | Stores register spills and large thread-private arrays.        |
| <b>Constant Memory</b> | Off-chip DRAM     | All threads / SMs        | Application        | Read-only; cached on-chip for fast broadcast access.           |
| <b>Global Memory</b>   | Off-chip DRAM     | <b>All threads / SMs</b> | <b>Application</b> | <b>Main memory; allocated by CPU, persists across kernels.</b> |

**Analysis of the Question Requirements:** The question asks: *"Which CUDA memory space is directly accessible by all Streaming Multiprocessors (SMs) and persists for the duration of the application?"*

- **Global Memory (Option 1):** Meets both criteria. Any thread running on any SM can read and write to global memory. Memory allocated via `cudaMalloc` persists until explicitly released via `cudaFree` or until host program termination.
- **L1 Cache (Option 2):** Private to an individual SM, hardware-managed, does not persist across application.
- **L2 Cache (Option 3):** A shared caching tier for DRAM transactions, not an addressable CUDA memory space.
- **Shared Memory (Option 4):** Private to each SM and scoped to a thread block; wiped upon block completion.

**Final Answer to Enter in Box: 1**

*Option 1: Global memory*

## Question 9: SIMT Model and Warp Divergence Summation

### Core Concepts: How GPUs Handle Conditional Branches

In NVIDIA's Single Instruction, Multiple Threads (SIMT) architecture, threads are executed in 32-thread warps. All 32 threads share a single instruction unit. If threads within a warp take different execution paths at a conditional branch (**if-else**), the warp serializes: it executes each active branch sequentially while masking off inactive threads. This is known as **warp divergence**.

### Problem Parameters:

- `blockDim.x = 32`, `gridDim.x = 1`.
- Exactly 1 warp launched, containing thread IDs:  $\text{tid} \in \{0, 1, 2, \dots, 31\}$ .
- Code:

```

1  if (tid % 4 == 1)
2      c[tid] = a[tid] - b[tid]; // Branch A
3  else
4      c[tid] = a[tid] + b[tid]; // Branch B
5

```

### Step-by-Step Identification and Summation:

1. **Determine which threads follow Branch A:** A thread follows Branch A if and only if  $\text{tid} \pmod{4} == 1$ . Testing all integers  $\text{tid}$  from 0 to 31:

Thread IDs satisfying  $\text{tid} \pmod{4} = 1$ :  $\{1, 5, 9, 13, 17, 21, 25, 29\}$ .

There are exactly 8 threads that take Branch A.

2. **Calculate the sum of these thread IDs:**

$$\text{Sum} = 1 + 5 + 9 + 13 + 17 + 21 + 25 + 29.$$

3. **Method 1: Pairwise Addition:**

$$(1 + 29) + (5 + 25) + (9 + 21) + (13 + 17) = 30 + 30 + 30 + 30 = \mathbf{120}.$$

4. **Method 2: Arithmetic Progression Formula:** First term  $a = 1$ , common difference  $d = 4$ , number of terms  $n = 8$ , last term  $l = 29$ :

$$S_n = \frac{n}{2}(a + l) = \frac{8}{2}(1 + 29) = 4 \times 30 = \mathbf{120}.$$

**Final Answer to Enter in Box: 120**

*The sum of the thread-IDs that follow Branch A is 120.*

## Question 10: Grid Dimensioning and Total Warps Launched

### Core Concepts: Grid Sizing Idiom and Warp Allocation

**The Ceiling Division Formula:** In CUDA, to ensure that at least  $N$  threads are launched when using blocks of size  $B$ , the grid size is calculated as:

$$\text{gridSize} = \frac{N + B - 1}{B} = \left\lceil \frac{N}{B} \right\rceil$$

**Warp Counting Rule:** A warp consists of 32 threads. Hardware schedules threads at the warp level. A warp is counted as launched as long as the block contains threads in that warp slot, even if some threads are inactive or masked.

### Problem Parameters:

- Problem size:  $N = 1 \ll 20 = 2^{20} = 1,048,576$ .
- Block size:  $\text{block.x} = 256$  threads.
- Warp size: 32 threads.

### Step-by-Step Derivation:

#### 1. Calculate the number of blocks in the grid:

$$\text{grid.x} = \frac{N + \text{block.x} - 1}{\text{block.x}} = \frac{1,048,576 + 256 - 1}{256} = \frac{1,048,831}{256} = 4096 \text{ blocks.}$$

(Note: Since  $1,048,576 = 4096 \times 256$ , the division is exact).

#### 2. Calculate warps launched per block:

$$\text{Warps per Block} = \frac{\text{block.x}}{\text{Warp size}} = \frac{256}{32} = 8 \text{ warps per block.}$$

#### 3. Calculate total warps launched across the entire grid:

$$\text{Total Warps Launched} = \text{grid.x} \times \text{Warps per Block}$$

$$\text{Total Warps Launched} = 4096 \text{ blocks} \times 8 \text{ warps/block} = 2^{12} \times 2^3 = 2^{15} = \mathbf{32,768} \text{ warps.}$$

#### 4. Alternative Direct Calculation from Total Threads:

$$\text{Total Threads} = 4096 \times 256 = 1,048,576 \text{ threads.}$$

$$\text{Total Warps} = \frac{\text{Total Threads}}{\text{Warp size}} = \frac{1,048,576}{32} = \mathbf{32,768} \text{ warps.}$$

**Note on Latency Parameters:** The problem description lists several latencies (400 cycles for memory loads/stores, 6 cycles for FP32 add). These are background performance characteristics typical of GPU architectures, but they do not affect the purely geometric calculation of total warps launched.

**Final Answer to Enter in Box: 32768**

*Total number of warps launched across the grid is 32768.*